

ChatGPT로 자율주행하기



Contents

1 코드로 핑키 자율주행 해보기

1.1 [로봇] pinky brinup 실행

1.2 [로봇] 내비게이션 실행

1.3 [PC] nav2_view.rviz 실행

1.4 2D Pose Estimate 버튼을 눌러 라이다와 맵 일치 시켜주기

1.5 PC 웹 브라우저를 새로 열고 아래 주소를 입력 후 접속

1.6 주피터 화면에서 new를 눌러 새 주피터 파일 생성

1.7 1. OpenAI API 설정

1.8 2. OpenAI API 연결 테스트

1.9 3. ROS2/Nav2 주행 라이브러리 불러오기

1.10 4. Nav2 실행 확인

1.11 5./cmd_velpublisher 준비

1.12 6. degree 각도를 quaternion으로 변환하는 함수

1.13 7. Nav2 goal pose 생성 함수

1.14 8. 주행 결과 변환 함수

1.15 9. 단일 목표 지점으로 이동하는 함수

1.16 10. 예시 좌표를 등록 위치로 만들기

1.17 11. 여러 좌표를 순서대로 주행하는 함수

1.18 12./cmd_vel직접 제어 함수

1.19 13. 정지 함수

1.20 14. 상태 확인 함수

1.21 15. OpenAI Tool 정의

1.22 16. function_call 실행 함수

1.23 17. 자연어 주행 명령 실행 함수

1.24 18. 실행 예시 1: 등록 위치로 이동

1.25 19. 실행 예시 2: 좌표를 직접 지정해서 이동

1.26 20. 실행 예시 3: 여러 좌표 순서대로 주행

1.27 21. 실행 예시 4: 앞으로 조금 이동

1.28 25. Nav2 종료

1.29 27. YOLO11 패키지와 카메라 라이브러리 불러오기

1.30 28. 카메라 프레임 한 장 가져오기

1.31 29. YOLO11n 모델 불러오기

1.32 30. 카메라 이미지 한 장에서 YOLO 검출하기

1.33 31. YOLO 검출 결과를 텍스트로 정리하기

1.34 32. YOLO 검출 결과 확인

1.35 33. YOLO 검출 결과를 바탕으로 OpenAI와 대화하기

1.36 34. 카메라를 새로 보고 바로 대화하기

1.37 35. 실행 예시 : 현재 화면을 새로 보고 답하기

1.38 36. YOLO 결과와 주행 명령을 함께 사용

ChatGPT로 자율주행하기

Exported from Confluence · <https://pinkwink.atlassian.net>

코드로 핑키 자율주행 해보기

[로봇] pinky brinup 실행

Code

```
1 ros2 launch pinky_bringup bringup_robot.launch.xml
```

```
pinky@raspi:~$ ros2 launch pinky_bringup bringup_robot.launch.xml
[INFO] [launch]: All log files can be found below /home/pinky/.ros
/log/2025-10-20-14-01-42-797058-raspi-5276
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [5280]
[INFO] [joint_state_publisher-2]: process started with pid [5281]
[INFO] [sllidar_node-3]: process started with pid [5282]
[INFO] [bringup-4]: process started with pid [5283]
[sllidar_node-3] [INFO] [1760936503.348025118] [sllidar_node]: SLL
idar running on ROS2 package SLLidar.ROS2 SDK Version:1.0.1, SLLID
AR SDK Version:2.1.0
[robot_state_publisher-1] [INFO] [1760936503.365403449] [robot_sta
te_publisher]: Robot initialized
```

[로봇] 내비게이션 실행

Code

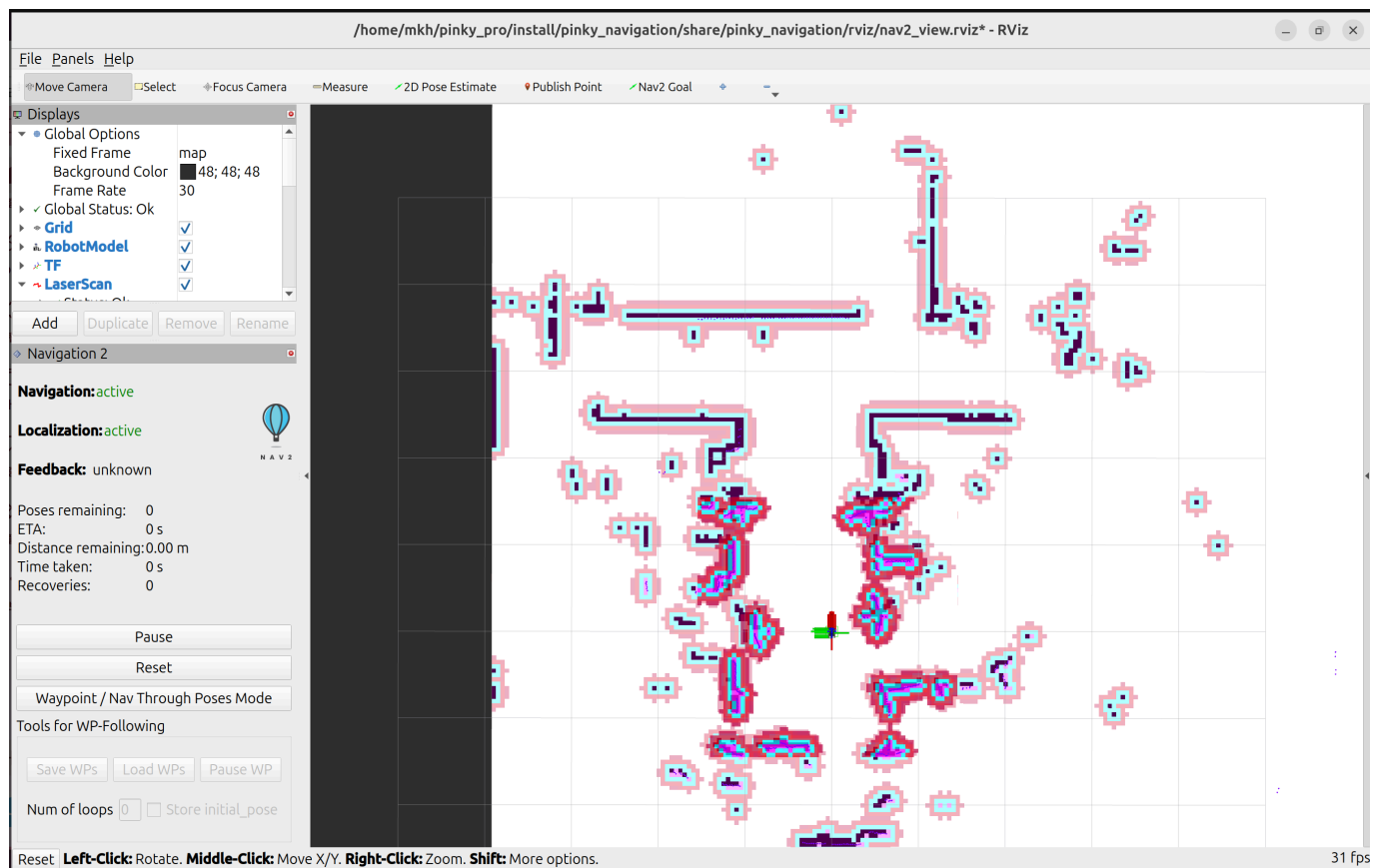
```
1 ros2 launch pinky_navigation bringup_launch.xml map:=<저장한 맵이름.yaml>
```

```
pinky@raspi:~$ ros2 launch pinky_navigation bringup_launch.xml map:=my_map
.yaml
[INFO] [launch]: All log files can be found below /home/pinky/.ros/log/2025-01-13-23-41-20-517658-raspi-2974
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [component_container_isolated-1]: process started with pid [2995]
[INFO] [cartographer_node-2]: process started with pid [2996]
[component_container_isolated-1] [INFO] [1736779281.944944101] [nav2_container]: Load Library: /opt/ros/jazzy/lib/libmap_server_core.so
[cartographer_node-2] [INFO] [1736779281.976917743] [cartographer logger]: I20250113 23:41:21.-2147483648 2996 configuration_file_resolver.cc:41] Found '/home/pinky/pinky_violet/install/pinky_cartographer/share/pinky_cartographer/params/nav2_cartographer_params.lua' for 'nav2_cartographer_params.lua'.
[cartographer_node-2] [INFO] [1736779281.977852329] [cartographer logger]: I20250113 23:41:21.-2147483648 2996 configuration_file_resolver.cc:41] Found '/opt/ros/jazzy/share/cartographer/configuration_files/map_builder.lua' for 'map_builder.lua'.
[cartographer_node-2] [INFO] [1736779281.977908700] [cartographer logger]: I20250113 23:41:21.-2147483648 2996 configuration_file_resolver.cc:41] Found '/opt/ros/jazzy/share/cartographer/configuration_files/map_builder.lua' for 'map_builder.lua'.
```

[PC] nav2_view.rviz 실행

Code

```
1 ros2 launch pinky_navigation nav2_view.launch.xml
```



2D Pose Estimate 버튼을 눌러 라이다와 맵 일치 시켜주기



PC 웹 브라우저를 새로 열고 아래 주소를 입력 후 접속

PC의 wi-fi가 pinky_**** 에 연결되어 있어야 한다



주피터 화면에서 new를 눌러 새 주피터 파일 생성



1. OpenAI API 설정

- **API_KEY** 에 본인의 OpenAI API Key를 입력하거나, dotenv 사용

```
[ ]: from openai import OpenAI
import json

API_KEY = "여기에_API_KEY_입력"

if API_KEY == "여기에_API_KEY_입력":
    raise ValueError("API_KEY 값을 본인의 OpenAI API Key로 바꾼 뒤 실행합니다.")

client = OpenAI(
    api_key=API_KEY
)

MODEL = "gpt-5.5"
```

```
[ ]: from openai import OpenAI
from dotenv import load_dotenv

load_dotenv(dotenv_path=".env", override=True)

client = OpenAI()

MODEL = "gpt-5.5"
```

2. OpenAI API 연결 테스트

- API Key가 정상인지 확인

```
[ ]: response = client.responses.create(  
    model=MODEL,  
    input="안녕. OpenAI API 연결 테스트야. 한 문장으로 짧게 답해줘."  
)  
  
print(response.output_text)
```

3. ROS2/Nav2 주행 라이브러리 불러오기

```
: import math  
import time  
  
import rclpy  
from rclpy.duration import Duration  
  
from geometry_msgs.msg import PoseStamped, Twist  
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult  
  
import tf_transformations
```

4. Nav2 실행 확인

```
if not rclpy.ok():  
    rclpy.init()  
  
nav = BasicNavigator()  
nav.waitUntilNav2Active()  
  
print("Nav2 is ready for use.")
```

5. /cmd_vel publisher 준비

```
cmd_vel_pub = nav.create_publisher(Twist, "cmd_vel", 10)  
  
print("/cmd_vel publisher is ready.")
```

6. degree 각도를 quaternion으로 변환하는 함수

```
def get_quaternion_from_yaw(yaw_degrees: float):  
    yaw_radians = math.radians(yaw_degrees)  
    quaternion = tf_transformations.quaternion_from_euler(0, 0, yaw_radians)  
    return quaternion
```


7. Nav2 goal pose 생성 함수

```
def make_goal_pose(x: float, y: float, yaw_deg: float = 0.0) -> PoseStamped:
    q = get_quaternion_from_yaw(yaw_deg)

    goal_pose = PoseStamped()
    goal_pose.header.frame_id = "map"
    goal_pose.header.stamp = nav.get_clock().now().to_msg()

    goal_pose.pose.position.x = float(x)
    goal_pose.pose.position.y = float(y)
    goal_pose.pose.position.z = 0.0

    goal_pose.pose.orientation.x = q[0]
    goal_pose.pose.orientation.y = q[1]
    goal_pose.pose.orientation.z = q[2]
    goal_pose.pose.orientation.w = q[3]

    return goal_pose
```

8. 주행 결과 변환 함수

```
def task_result_to_text(result) -> str:
    if result == TaskResult.SUCCEEDED:
        return "SUCCEEDED"

    if result == TaskResult.CANCELED:
        return "CANCELED"

    if result == TaskResult.FAILED:
        return "FAILED"

    return f"UNKNOWN: {result}"
```

9. 단일 목표 지점으로 이동하는 함수

```

def navigate_to_pose(x: float, y: float, yaw_deg: float = 0.0, timeout_sec: int = 300):
    goal_pose = make_goal_pose(x, y, yaw_deg)

    print(f"[Nav2] goToPose: x={x}, y={y}, yaw_deg={yaw_deg}")
    nav.goToPose(goal_pose)

    while not nav.isTaskComplete():
        rclpy.spin_once(nav, timeout_sec=1.0)

        feedback = nav.getFeedback()

        if feedback is not None:
            try:
                print(
                    "Distance remaining:",
                    round(feedback.distance_remaining, 3),
                    "m"
                )
            except Exception:
                pass

        if Duration.from_msg(feedback.navigation_time) > Duration(seconds=timeout_sec):
            nav.cancelTask()
            publish_zero_velocity()
            return {
                "success": False,
                "action": "navigate_to_pose",
                "result": "CANCELED",
                "message": f"{timeout_sec}초를 초과해서 주행을 취소했습니다.",
                "target": {
                    "x": x,
                    "y": y,
                    "yaw_deg": yaw_deg
                }
            }

    result = nav.getResult()
    result_text = task_result_to_text(result)

    return {
        "success": result == TaskResult.SUCCEEDED,
        "action": "navigate_to_pose",
        "result": result_text,
        "target": {
            "x": x,
            "y": y,
            "yaw_deg": yaw_deg
        }
    }

```

10. 예시 좌표를 등록 위치로 만들기

- 실제 환경에 맞게 `point1` , `point2` , `point3` 값을 수정

```

NAMED_LOCATIONS = {
    "point1": {"x": 1.1, "y": 1.0, "yaw_deg": 0},
    "point2": {"x": -1.4, "y": -1.0, "yaw_deg": 180},
    "point3": {"x": -1.35, "y": -0.30, "yaw_deg": 45},
}

def navigate_to_named_location(location_name: str, timeout_sec: int = 300):
    if location_name not in NAMED_LOCATIONS:
        return {
            "success": False,
            "action": "navigate_to_named_location",
            "message": f"등록되지 않은 위치입니다: {location_name}",
            "available_locations": list(NAMED_LOCATIONS.keys())
        }

    pose = NAMED_LOCATIONS[location_name]

    result = navigate_to_pose(
        x=pose["x"],
        y=pose["y"],
        yaw_deg=pose["yaw_deg"],
        timeout_sec=timeout_sec
    )

    result["location_name"] = location_name
    return result

```

11. 여러 좌표를 순서대로 주행하는 함수

```

def follow_waypoints(points: list, timeout_sec: int = 300):
    goal_pose_list = []

    for point in points:
        x = point["x"]
        y = point["y"]
        yaw_deg = point.get("yaw_deg", 0.0)

        goal_pose = make_goal_pose(x, y, yaw_deg)
        goal_pose_list.append(goal_pose)

    print(f"[Nav2] followWaypoints: {len(goal_pose_list)} points")

    # FollowWaypoints의 feedback에는 NavigateToPose와 달리 navigation_time이 없습니다.
    # 따라서 timeout은 feedback.navigation_time이 아니라 시작 시각 기준 경과 시간으로 계산합니다.
    start_time = nav.get_clock().now()

    nav.followWaypoints(goal_pose_list)

    while not nav.isTaskComplete():
        rclpy.spin_once(nav, timeout_sec=1.0)

        feedback = nav.getFeedback()

        if feedback is not None:
            try:
                print("Current waypoint:", feedback.current_waypoint + 1)
            except Exception:
                pass

        elapsed_sec = (nav.get_clock().now() - start_time).nanoseconds / 1_000_000_000

        if elapsed_sec > timeout_sec:
            nav.cancelTask()
            publish_zero_velocity()
            return {
                "success": False,
                "action": "follow_waypoints",
                "result": "CANCELED",
                "message": f"{timeout_sec}초를 초과해서 waypoint 주행을 취소했습니다.",
                "points": points
            }

    result = nav.getResult()
    result_text = task_result_to_text(result)

    return {
        "success": result == TaskResult.SUCCEEDED,
        "action": "follow_waypoints",
        "result": result_text,
        "points": points
    }

```

12. `/cmd_vel` 직접 제어 함수

Code

```
1  MAX_LINEAR_X = 0.12
2  MAX_ANGULAR_Z = 0.50
3  MAX_DIRECT_DRIVE_DURATION_SEC = 3.0
4  CMD_VEL_RATE_HZ = 10.0
5
6  DIRECT_DRIVE_SPEEDS = {
7      "slow": {
8          "linear_x": 0.05,
9          "angular_z": 0.25
10     },
11     "normal": {
12         "linear_x": 0.10,
13         "angular_z": 0.45
14     }
15 }
16
17
18 def clamp(value: float, min_value: float, max_value: float) -> float:
19     return max(min(float(value), max_value), min_value)
20
21
22 def publish_cmd_vel(linear_x: float = 0.0, angular_z: float = 0.0):
23     msg = Twist()
24
25     msg.linear.x = clamp(linear_x, -MAX_LINEAR_X, MAX_LINEAR_X)
26     msg.angular.z = clamp(angular_z, -MAX_ANGULAR_Z, MAX_ANGULAR_Z)
27
28     cmd_vel_pub.publish(msg)
29
30
31 def publish_zero_velocity():
32     publish_cmd_vel(0.0, 0.0)
33
34
35 def direct_drive_motion(motion: str, speed: str, duration_sec: float):
36     if motion not in ["forward", "backward", "turn_left", "turn_right"]:
37         return {
38             "success": False,
39             "action": "direct_drive_motion",
40             "message": f"지원하지 않는 동작입니다: {motion}"
41         }
42
43     if speed not in DIRECT_DRIVE_SPEEDS:
44         return {
45             "success": False,
46             "action": "direct_drive_motion",
47             "message": f"지원하지 않는 속도입니다: {speed}"
48         }
49
50     duration_sec = clamp(duration_sec, 0.1, MAX_DIRECT_DRIVE_DURATION_SEC)
51
```

```

52     linear_x = 0.0
53     angular_z = 0.0
54
55     if motion == "forward":
56         linear_x = DIRECT_DRIVE_SPEEDS[speed]["linear_x"]
57
58     elif motion == "backward":
59         linear_x = -DIRECT_DRIVE_SPEEDS[speed]["linear_x"]
60
61     elif motion == "turn_left":
62         angular_z = DIRECT_DRIVE_SPEEDS[speed]["angular_z"]
63
64     elif motion == "turn_right":
65         angular_z = -DIRECT_DRIVE_SPEEDS[speed]["angular_z"]
66
67     # Nav2 goal과 /cmd_vel 직접 제어가 동시에 동작하지 않도록 취소합니다.
68     try:
69         nav.cancelTask()
70     except Exception:
71         pass
72
73     steps = max(1, int(duration_sec * CMD_VEL_RATE_HZ))
74     sleep_time = 1.0 / CMD_VEL_RATE_HZ
75
76     print(
77         f"[cmd_vel] motion={motion}, speed={speed}, "
78         f"linear_x={linear_x}, angular_z={angular_z}, duration={duration_sec}"
79     )
80
81     for _ in range(steps):
82         publish_cmd_vel(linear_x=linear_x, angular_z=angular_z)
83         rclpy.spin_once(nav, timeout_sec=0.01)
84         time.sleep(sleep_time)
85
86     publish_zero_velocity()
87
88     return {
89         "success": True,
90         "action": "direct_drive_motion",
91         "motion": motion,
92         "speed": speed,
93         "duration_sec": duration_sec,
94         "linear_x": linear_x,
95         "angular_z": angular_z,
96         "message": "cmd_vel 직접 제어 명령을 실행했습니다."
97     }

```

13. 정지 함수

```
def stop_navigation():
    try:
        nav.cancelTask()
    except Exception:
        pass

    publish_zero_velocity()

    return {
        "success": True,
        "action": "stop_navigation",
        "message": "Nav2 주행 취소 및 cmd_vel 정지 명령을 보냈습니다."
    }
```

14. 상태 확인 함수

```
def get_navigation_status():
    return {
        "success": True,
        "action": "get_navigation_status",
        "available_locations": list(NAMED_LOCATIONS.keys()),
        "direct_drive_commands": [
            "forward",
            "backward",
            "turn_left",
            "turn_right"
        ]
    }
```

15. OpenAI Tool 정의

Code

```
1  tools = [  
2      {  
3          "type": "function",  
4          "name": "navigate_to_named_location",  
5          "description": "등록된 위치 이름으로 로봇을 이동시킨다. 사용 가능한 위치: point1, point  
6          2, point3",  
7          "parameters": {  
8              "type": "object",  
9              "properties": {  
10                 "location_name": {  
11                     "type": "string",  
12                     "enum": list(NAMED_LOCATIONS.keys()),  
13                     "description": "이동할 등록 위치 이름"  
14                 }  
15             },  
16             "required": ["location_name"],  
17             "additionalProperties": False  
18         },  
19         "strict": True  
20     },  
21     {  
22         "type": "function",  
23         "name": "navigate_to_pose",  
24         "description": "로봇을 map 좌표계 기준의 특정 x, y, yaw_deg 위치로 이동시킨다.",  
25         "parameters": {  
26             "type": "object",  
27             "properties": {  
28                 "x": {  
29                     "type": "number",  
30                     "description": "map 좌표계 기준 목표 x 좌표"  
31                 },  
32                 "y": {  
33                     "type": "number",  
34                     "description": "map 좌표계 기준 목표 y 좌표"  
35                 },  
36                 "yaw_deg": {  
37                     "type": "number",  
38                     "description": "목표 방향 각도. 단위는 degree"  
39                 }  
40             },  
41             "required": ["x", "y", "yaw_deg"],  
42             "additionalProperties": False  
43         },  
44         "strict": True  
45     },  
46     {  
47         "type": "function",  
48         "name": "follow_waypoints",  
49         "description": "여러 좌표를 순서대로 주행한다.",  
50         "parameters": {  
            "type": "object",
```



```

51         "properties": {
52             "points": {
53                 "type": "array",
54                 "description": "순서대로 이동할 좌표 목록",
55                 "items": {
56                     "type": "object",
57                     "properties": {
58                         "x": {"type": "number"},
59                         "y": {"type": "number"},
60                         "yaw_deg": {"type": "number"}
61                     },
62                     "required": ["x", "y", "yaw_deg"],
63                     "additionalProperties": False
64                 }
65             }
66         },
67         "required": ["points"],
68         "additionalProperties": False
69     },
70     "strict": True
71 },
72 {
73     "type": "function",
74     "name": "direct_drive_motion",
75     "description": "짧은 수동 이동을 위해 /cmd_vel을 직접 publish한다. 예: 앞으로 조금 가
기, 뒤로 조금 가기, 왼쪽으로 천천히 돌기, 오른쪽으로 천천히 돌기",
76     "parameters": {
77         "type": "object",
78         "properties": {
79             "motion": {
80                 "type": "string",
81                 "enum": ["forward", "backward", "turn_left", "turn_right"],
82                 "description": "수동 이동 방향"
83             },
84             "speed": {
85                 "type": "string",
86                 "enum": ["slow", "normal"],
87                 "description": "수동 이동 속도"
88             },
89             "duration_sec": {
90                 "type": "number",
91                 "description": "동작 시간. 단위는 초. 0.1초 이상 3.0초 이하로 사용한다."
92             }
93         },
94         "required": ["motion", "speed", "duration_sec"],
95         "additionalProperties": False
96     },
97     "strict": True
98 },
99 {
100     "type": "function",
101     "name": "stop_navigation",
102     "description": "현재 Nav2 주행을 취소하고 /cmd_vel을 0으로 만들어 로봇을 정지한다.",
103     "parameters": {

```

```

104         "type": "object",
105         "properties": {},
106         "required": [],
107         "additionalProperties": False
108     },
109     "strict": True
110 },
111 {
112     "type": "function",
113     "name": "get_navigation_status",
114     "description": "등록된 위치 목록과 직접 제어 명령 목록을 확인한다.",
115     "parameters": {
116         "type": "object",
117         "properties": {},
118         "required": [],
119         "additionalProperties": False
120     },
121     "strict": True
122 }
123 ]

```

16. function_call 실행 함수

```

def run_navigation_function(function_name: str, arguments: dict):
    if function_name == "navigate_to_named_location":
        return navigate_to_named_location(
            location_name=arguments["location_name"]
        )

    if function_name == "navigate_to_pose":
        return navigate_to_pose(
            x=arguments["x"],
            y=arguments["y"],
            yaw_deg=arguments["yaw_deg"]
        )

    if function_name == "follow_waypoints":
        return follow_waypoints(
            points=arguments["points"]
        )

    if function_name == "direct_drive_motion":
        return direct_drive_motion(
            motion=arguments["motion"],
            speed=arguments["speed"],
            duration_sec=arguments["duration_sec"]
        )

    if function_name == "stop_navigation":
        return stop_navigation()

    if function_name == "get_navigation_status":
        return get_navigation_status()

    return {
        "success": False,
        "message": f"알 수 없는 함수입니다: {function_name}"
    }

```

17. 자연어 주행 명령 실행 함수

Code

```
1 def ask_navigation(command: str):
2     instructions = f'''
3     너는 ROS2 Nav2 기반 핑키 자율주행 명령 해석기다.
4
5     규칙:
6     1. 등록된 위치로 이동하라는 명령에는 navigate_to_named_location을 사용한다.
7     2. 등록된 위치 이름은 {list(NAMED_LOCATIONS.keys())} 이다.
8     3. x, y, yaw_deg가 명확한 명령에는 navigate_to_pose를 사용한다.
9     4. 여러 좌표를 순서대로 이동하라는 명령에는 follow_waypoints를 사용한다.
10    5. "앞으로 조금 가", "뒤로 조금 가", "왼쪽으로 천천히 돌아", "오른쪽으로 천천히 돌아" 같은 짧은 수동
    이동에는 direct_drive_motion을 사용한다.
11    6. "조금"은 duration_sec 0.8~1.2 사이로 해석한다.
12    7. "천천히"는 speed="slow"로 해석한다.
13    8. 속도 표현이 없으면 speed="slow"를 우선 사용한다.
14    9. direct_drive_motion의 duration_sec는 3.0을 넘기지 않는다.
15    10. "멈춰", "정지", "취소", "stop"은 stop_navigation을 사용한다.
16    11. 가능한 위치나 상태를 묻는 명령에는 get_navigation_status를 사용한다.
17    12. 좌표, 위치, 이동 방향이 애매하면 함수를 호출하지 말고 확인 질문을 한다.
18    13. LCD, LED, 감정 행동은 사용하지 않는다.
19    '''
20
21    response = client.responses.create(
22        model=MODEL,
23        instructions=instructions,
24        input=command,
25        tools=tools,
26        tool_choice="auto",
27        parallel_tool_calls=False,
28        max_output_tokens=600,
29    )
30
31    function_calls = [
32        item for item in response.output
33        if item.type == "function_call"
34    ]
35
36    if not function_calls:
37        return response.output_text
38
39    function_outputs = []
40
41    for call in function_calls:
42        args = json.loads(call.arguments)
43
44        print(f"[모델이 선택한 함수] {call.name}")
45        print(f"[함수 인자] {args}")
46
47        result = run_navigation_function(call.name, args)
48
49        print(f"[함수 실행 결과] {result}")
50
```

```

51         function_outputs.append(
52             {
53                 "type": "function_call_output",
54                 "call_id": call.call_id,
55                 "output": json.dumps(result, ensure_ascii=False)
56             }
57         )
58
59     final_response = client.responses.create(
60         model=MODEL,
61         previous_response_id=response.id,
62         input=function_outputs,
63         tools=tools,
64         max_output_tokens=600,
65     )
66
67     return final_response.output_text

```

18. 실행 예시 1: 등록 위치로 이동

10번 참조

```

answer = ask_navigation("point1 위치로 이동해.")
print("\n[최종 답변]")
print(answer)

```

19. 실행 예시 2: 좌표를 직접 지정해서 이동

```

answer = ask_navigation("x=0.5, y=0 위치로 이동해. 방향은 0도로 해.")
print("\n[최종 답변]")
print(answer)

```

20. 실행 예시 3: 여러 좌표 순서대로 주행

```

: answer = ask_navigation("point1, point2, point3 순서로 이동해.")
  print("\n[최종 답변]")
  print(answer)

```

21. 실행 예시 4: 앞으로 조금 이동

```

: answer = ask_navigation("앞으로 조금 가.")
  print("\n[최종 답변]")
  print(answer)

```

25. Nav2 종료

```
# 실습 종료 시에만 실행합니다.  
nav.lifecycleShutdown()  
publish_zero_velocity()  
rclpy.shutdown()
```

27. YOLO11 패키지와 카메라 라이브러리 불러오기

```
from ultralytics import YOLO  
from PIL import Image  
from IPython.display import display, clear_output  
import io  
import os  
  
from pinkylib import Camera
```

28. 카메라 프레임 한 장 가져오기

```
def capture_one_frame(cam):  
    frame_bgr = cam.get_frame()  
    image_rgb = Image.fromarray(frame_bgr[:, :, ::-1])  
    return frame_bgr, image_rgb  
  
def test_camera_once():  
    cam = Camera()  
    cam.start()  
  
    try:  
        frame_bgr, image_rgb = capture_one_frame(cam)  
        display(image_rgb)  
        return frame_bgr  
  
    finally:  
        cam.close()
```

29. YOLO11n 모델 불러오기

```
yolo_model = YOLO("yolo11n.pt")  
print("YOLO model loaded.")
```

30. 카메라 이미지 한 장에서 YOLO 검출하기

```

def detect_once_from_camera(imgsz=320, conf=0.25, show=True):
    cam = Camera()
    cam.start()

    try:
        frame_bgr, image_rgb = capture_one_frame(cam)

        results = yolo_model.predict(
            source=frame_bgr,
            imgsz=imgsz,
            conf=conf,
            verbose=False
        )

        result = results[0]
        annotated_bgr = result.plot()
        annotated_rgb = Image.fromarray(annotated_bgr[:, :, ::-1])

        if show:
            print("[원본 이미지]")
            display(image_rgb)
            print("[YOLO 검출 결과]")
            display(annotated_rgb)

        return result, annotated_bgr

    finally:
        cam.close()

```

31. YOLO 검출 결과를 텍스트로 정리하기

Code

```
1  def summarize_yolo_result(result, max_objects=10):
2      detections = []
3
4      if result.bboxes is None or len(result.bboxes) == 0:
5          return {
6              "count": 0,
7              "detections": [],
8              "summary_text": "탐지된 물체가 없습니다."
9          }
10
11     # 이미지 크기 확인
12     image_height, image_width = result.orig_shape[:2]
13     center_x = image_width / 2
14
15     for i, box in enumerate(result.bboxes[:max_objects], start=1):
16         cls_id = int(box.cls[0])
17         name = yolo_model.names[cls_id]
18         conf = float(box.conf[0])
19
20         x1, y1, x2, y2 = [float(v) for v in box.xyxy[0]]
21         box_center_x = (x1 + x2) / 2
22
23         if box_center_x < center_x * 0.8:
24             horizontal_position = "left"
25         elif box_center_x > center_x * 1.2:
26             horizontal_position = "right"
27         else:
28             horizontal_position = "center"
29
30         detections.append(
31             {
32                 "index": i,
33                 "name": name,
34                 "confidence": round(conf, 3),
35                 "box_xyxy": [
36                     round(x1, 1),
37                     round(y1, 1),
38                     round(x2, 1),
39                     round(y2, 1),
40                 ],
41                 "horizontal_position": horizontal_position
42             }
43         )
44
45     lines = []
46
47     for item in detections:
48         lines.append(
49             f'{item["index"]}. {item["name"]} '
50             f'(confidence={item["confidence"]}, '
51             f'position={item["horizontal_position"]}, '
```



```

52         f'box={item["box_xyxy"]})'
53     )
54
55     return {
56         "count": len(detections),
57         "detections": detections,
58         "summary_text": "\n".join(lines)
59     }
60
61
62 def print_yolo_summary(summary):
63     print(f"탐지 개수: {summary['count']}")
64
65     if summary["count"] == 0:
66         print(summary["summary_text"])
67         return
68
69     print(summary["summary_text"])

```

32. YOLO 검출 결과 확인

```

last_yolo_result, last_yolo_annotated_bgr = detect_once_from_camera(
    imgsz=320,
    conf=0.25,
    show=True
)

last_yolo_summary = summarize_yolo_result(last_yolo_result)
print_yolo_summary(last_yolo_summary)

```

33. YOLO 검출 결과를 바탕으로 OpenAI와 대화하기

```
def ask_about_yolo_detection(question: str, yolo_summary: dict = None):
    if yolo_summary is None:
        if "last_yolo_summary" not in globals():
            return "먼저 YOLO 검출 셀을 실행해야 합니다."

        yolo_summary = last_yolo_summary

    instructions = '''
너는 로봇 카메라의 YOLO 객체 검출 결과를 학생에게 설명하는 보조자다.

규칙:
1. 답변은 한국어로 한다.
2. YOLO 검출 결과에 있는 물체만 근거로 답한다.
3. 검출 결과에 없는 물체를 보았다고 말하지 않는다.
4. confidence가 낮을 수 있으므로 단정하지 않고 "보입니다", "탐지되었습니다"처럼 표현한다.
5. 위치 정보는 left, center, right를 기준으로 설명한다.
6. 주행 명령은 직접 실행하지 않는다.
7. 위험 판단이 필요한 경우 사람, 의자, 컵, 병, 가방 등 장애물이 될 수 있는 물체를 언급한다.
'''

    detection_text = yolo_summary.get("summary_text", "탐지 결과가 없습니다.")

    prompt = f'''
사용자 질문:
{question}

YOLO 검출 결과:
{detection_text}
'''

    response = client.responses.create(
        model=MODEL,
        instructions=instructions,
        input=prompt,
        max_output_tokens=500,
    )

    return response.output_text
```

34. 카메라를 새로 보고 바로 대화하기

```
def look_and_talk(question: str, imgsz=320, conf=0.25, show=True):
    result, annotated_bgr = detect_once_from_camera(
        imgsz=imgsz,
        conf=conf,
        show=show
    )

    summary = summarize_yolo_result(result)

    print("[YOLO 검출 요약]")
    print_yolo_summary(summary)

    answer = ask_about_yolo_detection(
        question=question,
        yolo_summary=summary
    )

    return answer, summary, annotated_bgr
```

35. 실행 예시 : 현재 화면을 새로 보고 답하기

```
answer, current_yolo_summary, current_yolo_annotated_bgr = look_and_talk(
    "지금 로봇 앞에 무엇이 있는지 말해줘.",
    imgsz=320,
    conf=0.25,
    show=True
)

print("\n[OpenAI 답변]")
print(answer)
```

36. YOLO 결과와 주행 명령을 함께 사용

```
# 1. 카메라로 현재 상황 확인
answer, summary, annotated = look_and_talk("앞에 뭐가 보여?", show=True)
print(answer)

# 2. 안전하다고 판단되면 짧게 전진
answer = ask_navigation("앞으로 조금 가.")
print(answer)

# 3. 위험하다고 판단되면 정지
answer = ask_navigation("멈춰.")
print(answer)
```



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio